

~ - -zsh

~ - -zsh

~ - -zsh

~ - path_waitpid

~ - -

UW PICO 5.09

File: vars_dispatch.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include <unistd.h>
#define MAX_ARGS 64
#define BUF_SIZE 1024

// Function pointer definition for built-in command handler[cite: 22]
typedef int (*BuiltinHandler)(char **args);

typedef struct {
    const char *name;
    BuiltinHandler handler;
} BuiltinCommand;

// Forward declaration of built-in handlers[cite: 22]
int builtin_echo(char **args);
int builtin_setenv(char **args);
int builtin_help(char **args);

// Dispatch Table for In-Process Commands[cite: 22]
BuiltinCommand dispatch_table[] = {
    {"echo", builtin_echo},
    {"setenv", builtin_setenv},
    {"help", builtin_help},
    {NULL, NULL}
};

// Built-in Handler Implementations[cite: 22]
int builtin_echo(char **args) {
    for (int i = 1; args[i] != NULL; i++) {
        printf("%s%s", args[i], args[i + 1] ? " " : "");
    }
    printf("\n");
    return 0;
}

int builtin_setenv(char **args) {
    if (args[1] == NULL || args[2] == NULL) {
        printf("Usage: setenv <VAR> <VALUE>\n");
        return 1;
    }

    if (setenv(args[1], args[2], 1) == 0) {
        printf("Variable '%s' set to '%s'\n", args[1], args[2]);
    } else {

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

--zsh

--zsh

--zsh

-- path_waitpid

--

UW PICO 5.09

File: vars_dispatch.c

```

{"echo", builtin_echo},
{"setenv", builtin_setenv},
{"help", builtin_help},
{NULL, NULL}
};

// Built-in Handler Implementations[cite: 22]
int builtin_echo(char **args) {
    for (int i = 1; args[i] != NULL; i++) {
        printf("%s%s", args[i], args[i + 1] ? " " : "");
    }
    printf("\n");
    return 0;
}

int builtin_setenv(char **args) {
    if (args[1] == NULL || args[2] == NULL) {
        printf("Usage: setenv <VAR> <VALUE>\n");
        return 1;
    }

    if (setenv(args[1], args[2], 1) == 0) {
        printf("Variable '%s' set to '%s'\n", args[1], args[2]);
    } else {
        perror("setenv failed");
    }
    return 0;
}

int builtin_help(char **args) {
    printf("--- Shell Built-in Commands ---\n");
    printf("1. echo <text>          Print arguments\n");
    printf("2. setenv <var> <val> Set an environment variable\n");
    printf("3. help          Display this menu\n");
    printf("4. exit          Exit the shell\n");
    return 0;
}

// 1. VARIABLE EXPANSION ENGINE[cite: 23]
char* expand_variables(const char *input) {
    char *result = malloc(BUF_SIZE);
    int res_idx = 0;
    int len = strlen(input);
    int i = 0;

    while (i < len) {
        // Detect variable reference '$'[cite: 23]
        if (input[i] == '$') {

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

--zsh

--zsh

--zsh

-- path_waitpid

--

UW PICO 5.09

File: vars_dispatch.c

```

i++;
char var_name[128];
int var_idx = 0;

// Support nested/adjacent variables by reading variable name characters[cite: 24]
while (i < len && (isalnum(unsigned char)input[i]) || input[i] == '_') {
    var_name[var_idx++] = input[i++];
}
var_name[var_idx] = '\0';

if (var_idx > 0) {
    // Lookup variable; handle undefined variables safely[cite: 24]
    char *val = getenv(var_name);
    if (val) {
        while (*val && res_idx < BUF_SIZE - 1) {
            result[res_idx++] = *val++;
        }
    }
    result[res_idx++] = '$'; // Isolated '$' symbol[cite: 24]
}
} else {
    result[res_idx++] = input[i++];
}
}

result[res_idx] = '\0';
return result;
}

// 2. DISPATCH TABLE EXECUTION[cite: 24]
int execute_builtin(char **args) {
    if (args[0] == NULL) return 0;

    for (int i = 0; dispatch_table[i].name != NULL; i++) {
        if (strcmp(args[0], dispatch_table[i].name) == 0) {
            // Execute in-process handler[cite: 24]
            return dispatch_table[i].handler(args);
        }
    }
    return -1; // Not a built-in command[cite: 25]
}

int main() {
    char input_buf[BUF_SIZE];

    while (1) {
        printf("\nDispatch Shell > ");

```

^G Get Help
^X Exit

^O WriteOut
^J Justify

^R Read File
^W Where is

^Y Prev Pg
^V Next Pg

^K Cut Text
^U UnCut Tex

```
~ -- zsh | ~ -- zsh | ~ -- zsh | ~ -- path_waitpid | ~ --
UW PICO 5.09 File: vars_dispatch.c

    if (!fgets(input_buf, sizeof(input_buf), stdin)) break;

    input_buf[strcspn(input_buf, "\n")] = 0; // Strip newline[cite: 25]
    if (strcmp(input_buf, "exit") == 0) break;

    // Step A: Expand Variable References[cite: 25]
    char *expanded_input = expand_variables(input_buf);
    printf("[Debug Expansion] Original: \"%s\" -> Expanded: \"%s\"\n", input_buf, expanded_input);

    // Step B: Tokenize Input[cite: 25]
    char *args[MAX_ARGS];
    int arg_count = 0;
    char *token = strtok(expanded_input, " ");

    while (token != NULL && arg_count < MAX_ARGS - 1) {
        args[arg_count++] = token;
        token = strtok(NULL, " ");
    }
    args[arg_count] = NULL;

    // Step C: Dispatch In-Process Built-ins or handle external fallback[cite: 25]
    if (arg_count > 0) {
        int result = execute_builtin(args);
        if (result == -1) {
            printf("[Dispatch] '%s' is an external/invalid command (not in dispatch table).\n", args[0]);
        }
    }

    free(expanded_input);
}

return 0;
}
```

⌘ Get Help

⌘ Exit

⌘ WriteOut

⌘ Justify

⌘ Read File

⌘ Where is

⌘ Prev Pg

⌘ Next Pg

⌘ Cut Text

⌘ UnCut Text

```
~ -- zsh | ~ -- zsh | ~ -- zsh | ~ -- path_waitpid | ~ --
[medikondarakesh@Medikondas-MacBook-Air ~ % nano vars_dispatch.c
medikondarakesh@Medikondas-MacBook-Air ~ % gcc -g -Wall vars_dispatch.c -o vars_dispatch
./vars_dispatch

Dispatch Shell > echo User: $USER Home: $HOME
[Debug Expansion] Original: "echo User: $USER Home: $HOME" -> Expanded: "echo User: medikondarakesh Home: /Users/medikondarakesh"
User: medikondarakesh Home: /Users/medikondarakesh

Dispatch Shell > echo Pre_$UNDEFINED_VAR_NAME_Post $USER
[Debug Expansion] Original: "echo Pre_$UNDEFINED_VAR_NAME_Post $USER" -> Expanded: "echo Pre_ medikondarakesh"
Pre_ medikondarakesh

Dispatch Shell > setenv MY_ROLE Developer
[Debug Expansion] Original: "setenv MY_ROLE Developer" -> Expanded: "setenv MY_ROLE Developer"
Variable 'MY_ROLE' set to 'Developer'

Dispatch Shell > help
[Debug Expansion] Original: "help" -> Expanded: "help"
--- Shell Built-in Commands ---
1. echo <text>          Print arguments
2. setenv <var> <val> Set an environment variable
3. help              Display this menu
4. exit              Exit the shell

Dispatch Shell > custom_unknown_cmd
[Debug Expansion] Original: "custom_unknown_cmd" -> Expanded: "custom_unknown_cmd"
[Dispatch] 'custom_unknown_cmd' is an external/invalid command (not in dispatch table).

Dispatch Shell > exit
medikondarakesh@Medikondas-MacBook-Air ~ %
```